

Worksharing Constructs

Victor Eijkhout

2026 PCSE

What is worksharing again?

- The `omp parallel` creates a team of threads.
- Now you need to distributed work among them.
- Already seen: `for`, `do`
- Similar: `sections`
- Not obvious: `single`
- Fortran only: `workshare` works with array notation
- Story in itself: `task`

Sections

Independent separate calculations:

```
1  double fx = f(x), gx = g(x), hx = h(x);
2  ..... fx ... gx ... hx ....

1 // sections.c
2 #pragma omp parallel sections
3 {
4     #pragma omp section
5     fx = f(1.);
6     #pragma omp section
7     gx = g(1.);
8     #pragma omp section
9     hx = h(1.);
10 }
11 float s = fx+gx+hx;
```

Sections with reduction

Same code, but with reduction:

```
1 float s=0;
2 #pragma omp parallel sections reduction(+:s)
3 {
4     #pragma omp section
5     s += f(1.);
6     #pragma omp section
7     s += g(1.);
8     #pragma omp section
9     s += h(1.);
10 }
```

Single

```
1 int a;
2 #pragma omp parallel
3 {
4     #pragma omp for reduction(+:x)
5     for ( ... i ... )
6         x += // something
7     #pragma omp single
8         y = fx();
9     #pragma omp for
10    for ( ... i ... )
11        z[i] = ... i ... y ...
12 }
```

- Is executed by a single thread.
- Has implicit barrier, so the result is available to all threads after.

Masked/Master

- Masked/Master: single thread execution
- Not a workshare, so no barrier

```
1 int a;  
2 #pragma omp parallel  
3 {  
4     // stuff  
5     #pragma omp master  
6         printf("...",a);  
7     // stuff  
8 }
```

(*masked* introduced in 5.1; *master* deprecated in 5.2)

Master wrongly used

```
1 int a;  
2 #pragma omp parallel  
3 {  
4     #pragma omp master  
5     a = f(); // some computation  
6     // WRONG: 'a' may not be available  
7     #pragma omp sections  
8     // various different computations using 'a'  
9 }
```

Exercise 1

What is the difference between this approach and how the same computation would be parallelized in MPI?